Estructura de Datos y Algoritmos

Estructuras de datos

VIGILADA MINEDUCACIÓN

Facultad de
Ingeniería

Universidad de La Sabana



Objetivos

- ✓ Comprender los tipos de algoritmos que hay
- ✓ Poner en práctica los conocimientos adquiridos en la clase

Universidad de La Sabana | Facultad de Ingeniería


Edificio K

¿Qué tipos de algoritmos existen?

En el vasto mundo de la programación y la ciencia de la computación, los algoritmos son la esencia que impulsa la eficiencia y el rendimiento de nuestros programas. Estos algoritmos pueden clasificarse en diferentes categorías, cada una con su propio conjunto de características y aplicaciones específicas.



Fuerza bruta

- La fuerza bruta es la estrategia más sencilla y directa.
- Explora todas las posibles soluciones y selecciona la óptima.
- Aunque puede carecer de eficiencia para problemas grandes, es un enfoque valioso para solucionar desafíos más pequeños.



Función de recurrencia

Una **función de recurrencia** es una ecuación que define una sucesión de forma recursiva. Esto significa que cada término de la sucesión se define en función de uno o más de los términos anteriores.

En otras palabras, en lugar de dar una fórmula explícita para cada término, la función de recurrencia describe cómo se puede calcular cada término a partir de los que ya se conocen.

Ejemplo:

La sucesión de Fibonacci se define mediante la siguiente función de recurrencia:

```
F(n) = {
    0 si n = 0
    1 si n = 1
    F(n-1) + F(n-2) si n > 1
}
```

En este caso, los dos primeros términos son 0 y 1, y cada término posterior se calcula sumando los dos términos anteriores.



Función de recurrencia

Aplicaciones:
Las funciones de recurrencia se utilizan en muchas áreas de las matemáticas y la informática, como:

- **Análisis de algoritmos**: para calcular la complejidad temporal de algoritmos recursivos.
- **Teoría de números**: para estudiar las propiedades de sucesiones de números, como la sucesión de Fibonacci.
- **Ciencia computacional**: para modelar fenómenos que se repiten de forma regular, como el crecimiento de una población.



Función de recurrencia

Ventajas:

Las funciones de recurrencia son una herramienta poderosa para definir sucesiones de forma concisa y elegante. Además, permiten modelar fácilmente fenómenos que se repiten de forma regular.

Desventajas:

En algunos casos, puede ser difícil encontrar una fórmula explícita para los términos de una sucesión definida por una función de recurrencia. Además, las funciones de recurrencia pueden ser difíciles de analizar matemáticamente.



Problema - 00003

Dada una cadena de caracteres, tu tarea es encontrar la longitud de la subcadena palindrómica más larga. Una subcadena palindrómica es aquella que se lee igual hacia adelante que hacia atrás. Implementa un algoritmo de fuerza bruta para resolver este problema.

Entrada:

Una cadena de caracteres (longitud máxima: 1000).

Salida:

La longitud de la subcadena palindrómica más larga encontrada en la cadena de entrada.

Entrada: babad	Salida: 3



Problema - 00004

Dada una lista de números enteros, tu tarea es encontrar el subarreglo (contiguo) con la mayor suma posible. Implementa un algoritmo de fuerza bruta para resolver este problema.

Entrada:

Una lista de números enteros, donde el primer número n ($1 \leq n \leq 1000$) indica la longitud de la lista, seguido por n números enteros separados por espacios.

Salida:

La suma máxima obtenida al considerar cualquier subarreglo contiguo en la lista.

Entrada:	Salida:
8 -2 1 -3 4 -1 2 1 -5 3 -2 1 4	6 5

Explicación: El subarreglo [4, -1, 2, 1] tiene la mayor suma (4 - 1 + 2 + 1 = 6).



Dinámica

- Los algoritmos dinámicos dividen un problema en subproblemas y resuelven cada subproblema solo una vez, almacenando los resultados para su uso futuro.
- Son ideales para problemas que exhiben subestructuras óptimas y solapan subproblemas.



Programación Dinámica

Ventajas:

- La programación dinámica puede mejorar significativamente el tiempo de ejecución de algoritmos para resolver problemas de optimización.
- Es una técnica versátil que se puede aplicar a una amplia variedad de problemas.

Desventajas:

- La programación dinámica puede ser difícil de implementar para algunos problemas.
- Puede requerir una gran cantidad de memoria para almacenar las soluciones de los subproblemas.



Programación Dinámica

Programación dinámica determinística:

En este tipo de programación dinámica, los resultados de cada subproblema son completamente predecibles y no dependen de ningún factor aleatorio.

La función de recurrencia que se utiliza para calcular la solución óptima es determinista, es decir, siempre produce el mismo resultado para los mismos valores de entrada.

Programación dinámica probabilística:

En este tipo de programación dinámica, los resultados de cada subproblema no son completamente predecibles y dependen de algún factor aleatorio.

La función de recurrencia que se utiliza para calcular la solución óptima es probabilística, es decir, puede producir diferentes resultados para los mismos valores de entrada, dependiendo de la distribución de probabilidad de los factores aleatorios.



Programación Dinámica

Tabulación:

- Enfoque bottom-up, también conocido como iterativo.
- Se comienza resolviendo los casos más pequeños y se almacenan los resultados en una tabla.
- Se utiliza un bucle para iterar desde los casos más pequeños hasta el caso deseado.
- Menos propenso a problemas de desbordamiento de pila en comparación con la recursión pura.

Memorización:

- Enfoque top-down, también conocido como recursivo con memorización.
- Se utiliza la recursión, pero con la diferencia clave de que los resultados intermedios se almacenan en una estructura de datos (generalmente un diccionario o matriz) para evitar recalcularlos.
- En cada llamada recursiva, se verifica si ya se ha calculado el resultado y se utiliza ese valor almacenado en lugar de recalcularlo.
- Puede ser más fácil de implementar en algunos casos debido a la estructura recursiva.
- Puede ser más eficiente en términos de tiempo que la recursión pura debido a la evitación de cálculos redundantes.



Problema - 00005

Minimizar el Cambio

Supongamos que tienes un sistema de monedas de diferentes denominaciones (pueden ser centavos, euros, etc.). Tu tarea es implementar un algoritmo que, dado un monto total que debes devolver como cambio, determine la cantidad mínima de monedas que necesitas para lograr el cambio exacto.

Escribe una función que tome como entrada el monto total y una lista de valores de monedas disponibles. La función deberá devolver la cantidad mínima de monedas necesarias para realizar el cambio.

Por ejemplo, si las monedas disponibles son `[1, 5, 10, 25]` (centavos) y el monto total es `63` centavos, la respuesta debería ser `6` monedas (dos de 25 centavos, una de 10 centavos y una de 1 centavo).

Entrada:
- Un entero representando el monto total.
- Una lista de enteros representando los valores de las monedas disponibles.

Salida:
- Un entero representando la cantidad mínima de monedas necesarias para el cambio.

Entrada:	**Salida**:
63 1 5 10 25	6



Problema - 00006

Dada una mochila con una capacidad C y un conjunto de n objetos, cada uno con un peso W_i y un valor V_i, encontrar el subconjunto de objetos que maximize el valor total de la mochila sin superar su capacidad.

Entrada:

Las siguientes tres líneas por caso contienen: en la primera línea un numero **C** donde es la capacidad de la mochila, en la siguiente línea contiene los pesos **W** de un numero **n** de objetos, y en la tercera línea contiene los valores **V** un numero n de objetos.

Salida:

Los Valores de los objetos que pueden llevar en la maleta sin superar la capacidad de la mochila

Entrada:	**Salida**:
8 3 2 4 2 5 8	2 8



Voraces (Greedy)

- Los algoritmos voraces toman decisiones óptimas en cada paso con la esperanza de llegar a una solución global óptima.
- Son útiles en problemas de optimización donde se toman decisiones locales para alcanzar el mejor resultado general.



Divide y Vencerás

- Divide y vencerás divide un problema en subproblemas más pequeños, resuelve cada subproblema y combina las soluciones.
- Es particularmente eficaz en problemas que se pueden dividir fácilmente y cuya solución óptima se puede combinar de manera eficiente.



Ventajas y Desventajas de Divide y Vencerás

Ventajas:

- Favorece el uso de la recursividad, lo que hace que el diseño de algoritmos sea más intuitivo.
- Permite mejoras importantes en la eficiencia (por ejemplo, mejora de $O(n^2)$ a $O(n \log n)$ en ordenamientos).
- Buena elección para paralelización en arquitecturas multicore.

Desventajas:

- Uso intensivo de la pila de llamadas puede causar desbordamientos si no se maneja bien.
- En algunos casos, la recombinación de resultados puede ser costosa.
- No todos los problemas se pueden dividir de forma efectiva.



Problema - 00007

Dado un arreglo de números enteros, encuentra el elemento máximo y el mínimo utilizando el enfoque de divide y vencerás.

Entrada:
Una lista de n números enteros.

Salida:
Una tupla con el valor mínimo y máximo encontrados.

Entrada:	**Salida**:
[4, 2, 7, 1, 9, 3]	(1, 9)



El Teorema Maestro

El teorema maestro sirve para resolver relaciones recursivas de la forma:

$$T(n) = a\,T\left(\frac{n}{b}\right) + f(n) \quad \text{donde} \quad a \geq 1, b > 1$$

En la aplicación de análisis de algoritmos recursivos, las constantes y funciones toman los siguientes significados:

- $\boldsymbol{n}$ es el tamaño del problema a resolver.
- $\boldsymbol{a}$ es el número de subproblemas en la recursión.
- $\boldsymbol{n/b}$ el tamaño de cada subproblema. (Aquí se asume que todos los subproblemas tienen el mismo tamaño)
- $f\ (n)$ es el costo del trabajo hecho fuera de las llamadas recursivas, que incluye el costo de la división del problema y el costo de unir las soluciones de los subproblemas.



El Teorema Maestro

- **Caso 1:**

Si $f(n) = O(n^{\log_b a - \epsilon})$ para algún $\epsilon > 0$,

entonces:

$$T(n) = \Theta(n^{\log_b a})$$

- *El trabajo principal está en la recursión.*



El Teorema Maestro

- **Caso 2:**

Si $f(n) = \Theta(n^{\log_b a})$,

entonces:

$$T(n) = \Theta(n^{\log_b a} \cdot \log n)$$

- *El trabajo se distribuye equitativamente.*



El Teorema Maestro

- **Caso 3:**

Si $f(n) = \Omega(n^{\log_b a+\epsilon})$ para algún $\epsilon > 0$ y se cumple la condición de regularidad:

$$a \cdot f\left(\frac{n}{b}\right) \leq c \cdot f(n) \quad \text{para algún } c < 1,$$

entonces:

$$T(n) = \Theta(f(n))$$

- *El trabajo principal está en la parte no recursiva.*



¿Cuándo usarlo y cuándo no?

✅ **Úsalo cuando** tu recurrencia tiene la forma exacta:

$$T(n) = aT(n/b) + f(n)$$

❌ **No se puede usar cuando:**

- Los subproblemas son de tamaños distintos
- Hay dependencia entre subproblemas
- La división del problema no es uniforme



Backtracking

- El enfoque de vuelta atrás se utiliza para buscar todas las soluciones posibles para un problema.
- Explora cada opción y retrocede cuando encuentra una solución inviable.
- Es eficaz para problemas combinatorios y de optimización.

Facultad de
Ingeniería



Universidad de
La Sabana